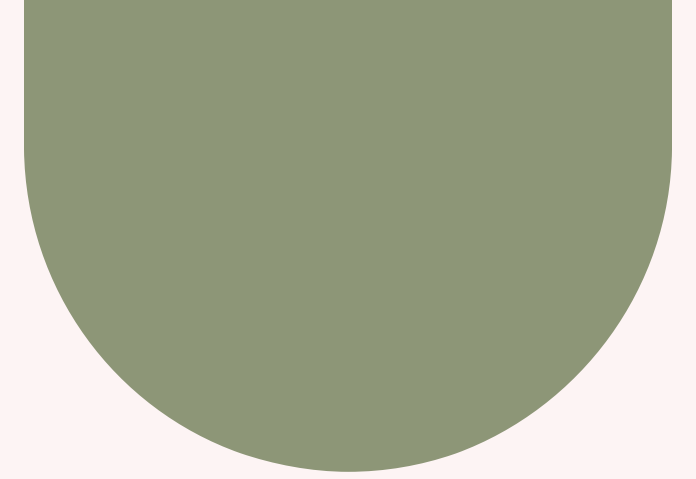
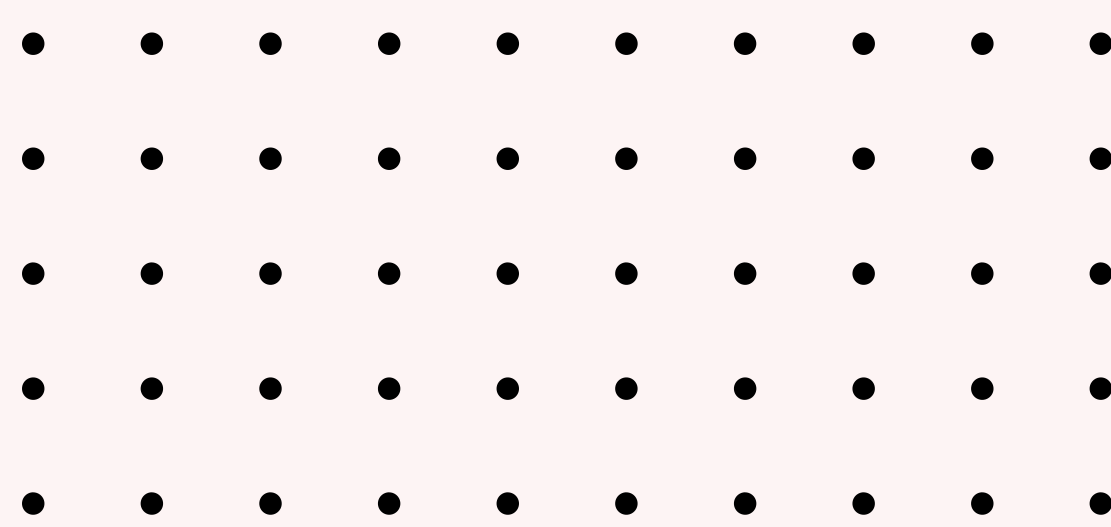
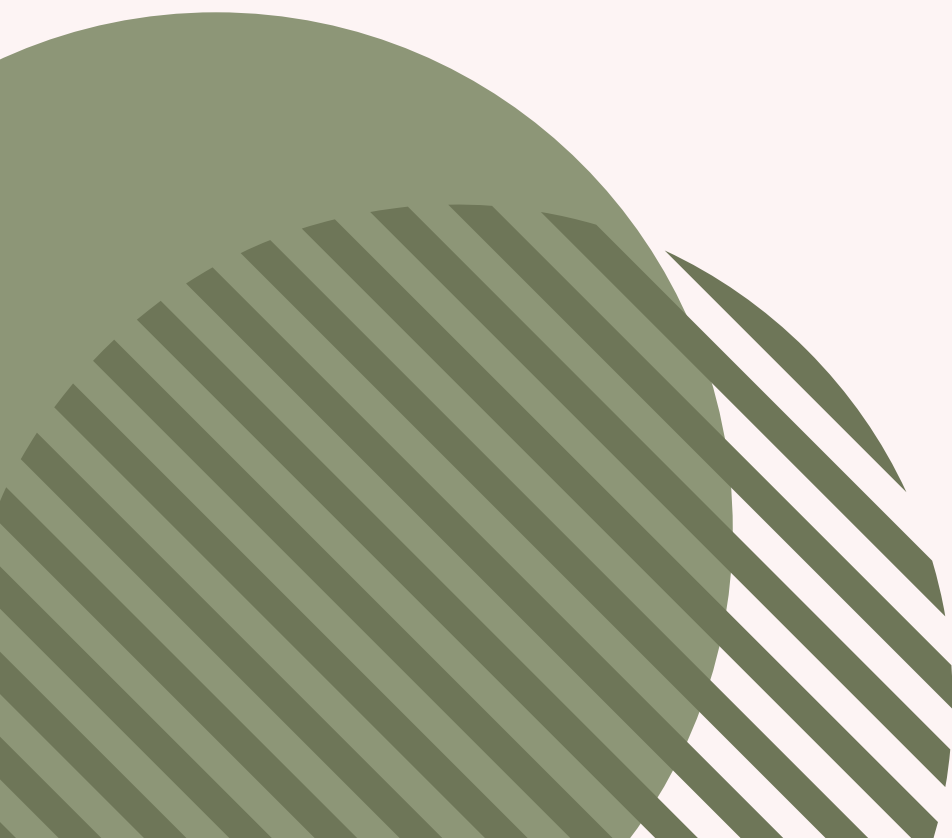




OOP Group Project

By Group 75 from Section 4



About Our Application

Our application: *Green Echoes*, is a highly regarded educational platform which focused on shedding light on the intricacies of climate change. It is a sophisticated tool that offers users a valuable resource, enabling them to develop a comprehensive understanding of the climate crisis and empowering them to actively engage in sustainable practices.

This educational platform dedicated to climate change provides a wealth of information and resources to help users gain a deeper understanding of the complexities surrounding the climate crisis.



About Our Features

Our application divided into few sections/features...

- 1) Main Page
- 2) User Authentication
- 3) Information Hub
- 4) Carbon Footprint Calculator
- 5) Community and Engagement Collaboration
- 6) Real Time Environment Alerts
- 7) Quiz System
- 8) Feedback and Rating System



Main Page

- Main Page serves as the central hub where users can access and navigate through various features offered by our application
- It empowers users by providing a brief overview of climate change mitigation, adaptation strategies, and the significance of early warning systems, fostering a deeper understanding of sustainable practices




```
package oop.group.project;
import java.util.Scanner;

public class MainPage {
    public static void main(String[] args) {
        System.out.println(x: "Welcome to XXX!");
        System.out.println(x: "Your journey towards a more sustainable lifestyle begins here. Let's make every choice count.");
        System.out.println(x: "Our goal is to empower individuals with knowledge and resources, fostering a deep understanding of climate change mitigation,
        Scanner sc = new Scanner(System.in);
```

Scanner

- > a class that used to capture user input from the console
- > it is a part of 'java.util' package

'sc' is a new instance of the 'Scanner' to read input from the standard input steam ('System.in)

```
Welcome to Green Echoes!
```

```
Your journey towards a more sustainable lifestyle begins here. Let's make every choice count.
```

```
Our goal is to empower individuals with knowledge and resources, fostering a deep understanding of
climate change mitigation, adaptation strategies, and the importance of early warning systems.
```

```
int choice = 0;

do {
    System.out.println(x: "-----");
    System.out.println(x: "These are the features we provide:");
    System.out.println(x: "1. Login");
    System.out.println(x: "2. Information Hub");
    System.out.println(x: "3. Carbon Footprint Calculator");
    System.out.println(x: "4. Community Engagement and Collaboration");
    System.out.println(x: "5. Real-time Environmental Alerts");
    System.out.println(x: "6. Quiz System");
    System.out.println(x: "7. Feedback and Rating System");
    System.out.println(x: "-----");
    System.out.print(s: "Enter the number of your choice: ");

```

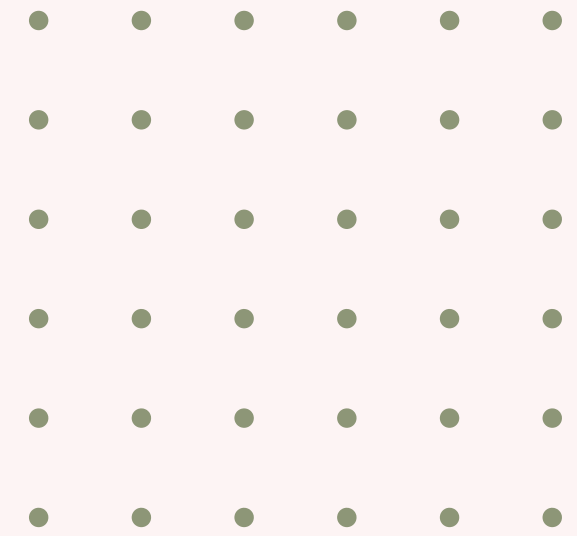
```
} while (choice < 1 || choice > 7);
```

int choice = 0

The variable choice is initialized with a value (e.g. 0) before entering the loop, this is to ensure that the loop executes at least once

do-while loop

prints out our features first and then only checks the loop condition



```
-----
These are the features we provide:
1. Login
2. Information Hub
3. Carbon Footprint Calculator
4. Community Engagement and Collaboration
5. Real-time Environmental Alerts
6. Quiz System
7. Feedback and Rating System
-----
Enter the number of your choice:
```

```

if (sc.hasNextInt()) {

    choice = sc.nextInt();

    switch (choice) {
        case 1:
            UserAuthentication login = new UserAuthentication();
            login.login();
            break;
        case 2:
        case 3:
        case 4:
        case 5:
        case 6:
        case 7:
            if (choice != 1) {
                handleLogin(sc);
            }
            break;
        default:

            System.out.println(x: "Please enter from 1 to 7!");
    }
} else {
    sc.next();
    System.out.println(x: "Invalid input. Please enter a valid integer option.");
}

} while (choice < 1 || choice > 7);

```

if (sc.hasNextInt())

this method is in an **if** statement, it checks whether the next input is a valid integer.

if it returns true, the code inside the if block will be executed

if false, it prompts the user to enter a valid input using the '**else**' statement

switch (choice)

this method evaluates the value of 'choice', then directs to the matching '**case**'

```

if (sc.hasNextInt()) {

    choice = sc.nextInt();

    switch (choice) {
        case 1:
            UserAuthentication login = new UserAuthentication();
            login.login();
            break;
        case 2:
        case 3:
        case 4:
        case 5:
        case 6:
        case 7:
            if (choice != 1) {
                handleLogin(sc);
            }
            break;
        default:

            System.out.println(x: "Please enter from 1 to 7!");
    }
} else {
    sc.next();
    System.out.println(x: "Invalid input. Please enter a valid integer option.");
}

} while (choice < 1 || choice > 7);

```

case 1 :

When the user enter the option 1, the program initiates the UserAuthentication class, leading the execution of the login method

case 2-7:

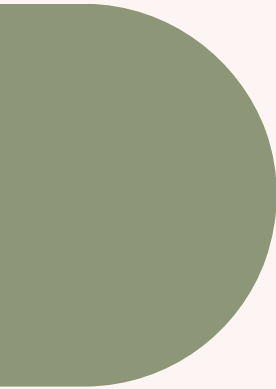
If the user enter the option 2 to 7, program directs to the handleLogin method

break;

used as the last statement in each case, so that if the condition is met, the other statements won't be executed

default;

If the entered number does not match any of the specified cases, the code inside the 'default' will be executed, prompting the user to enter a number from 1 to 7.



```
private static void handleLogin(Scanner sc) {  
    System.out.println(x: "Please login to proceed.");  
    UserAuthentication login = new UserAuthentication();  
    login.login();  
}
```

The first login is the instance of the UserAuthentication class, the second login is the method from the UserAuthentication class

The handleLogin method is called when the user enters a choice from 2 to 7 in the Main Page. It prompts the user to login.

User Authentication

- This is where we verify the identity of a user before granting access to our application
- A secure login system safeguards against unauthorized access and protects sensitive information



Behind the scenes

```
package oop.group.project;
```

```
public class UserAuthentication {  
    public static void login() {  
        LoginGUI LoginFrame = new LoginGUI();  
        LoginFrame.setVisible(b: true);  
        LoginFrame.pack();  
        LoginFrame.setLocationRelativeTo(c: null);  
    }  
}
```

login() method :

>used to instantiate the 'LoginGUI' class

setVisible(true) method :

>makes the frame visible on our screen

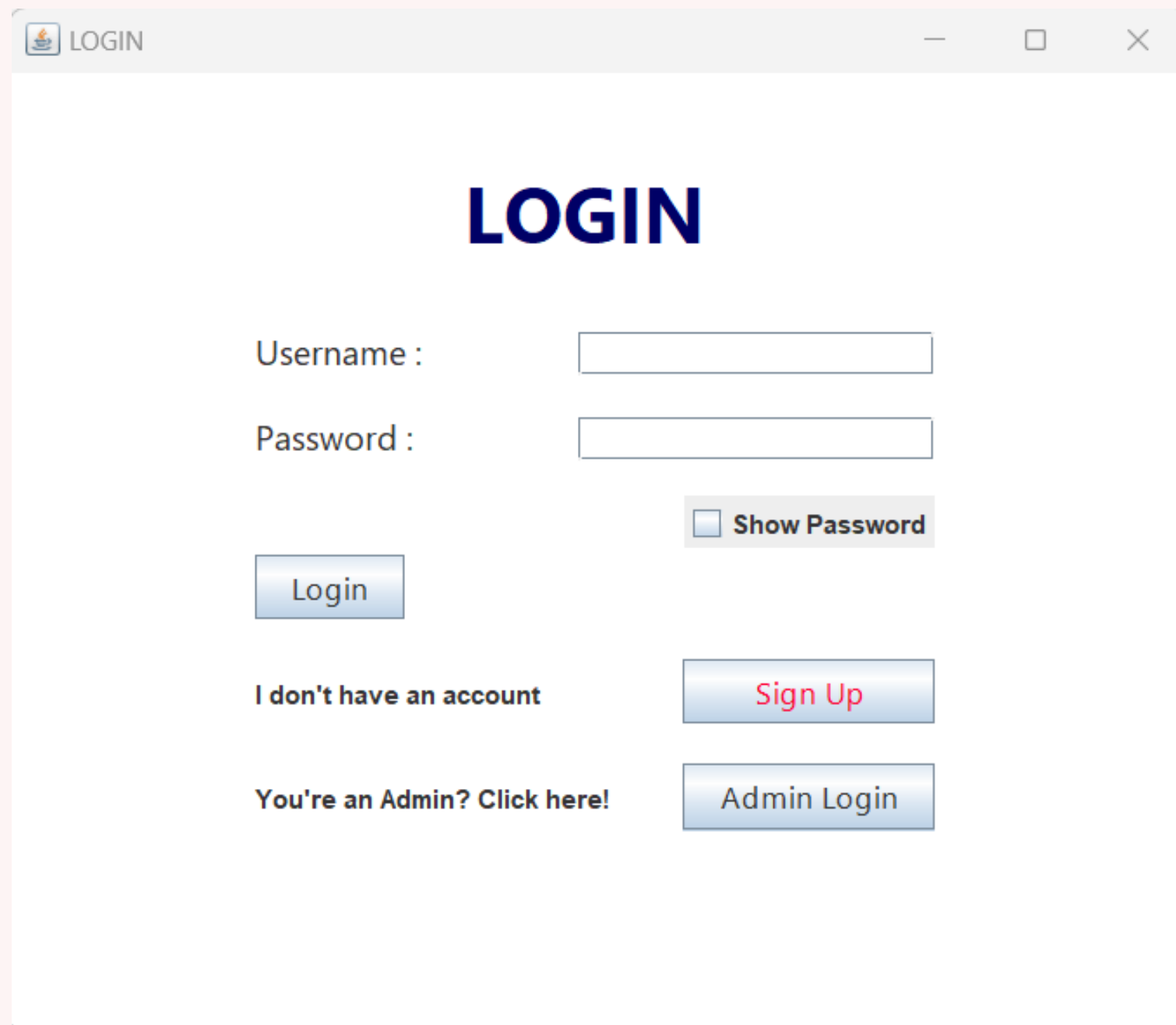
setLocationRelativeTo(null) method :

>positions the frame at the center of the screen



This is where we bring our user to our login system

Login GUI



The screenshot shows a Java Swing window titled "LOGIN". Inside the window, the word "LOGIN" is displayed in a large, bold, dark blue font. Below it, there are two text input fields for "Username :" and "Password :". To the right of the password field is a checkbox labeled "Show Password". Below these fields is a blue "Login" button. At the bottom left, there is a link "I don't have an account" and a blue "Sign Up" button. At the bottom right, there is a link "You're an Admin? Click here!" and a blue "Admin Login" button.

```
package oop.group.project;
import javax.swing.JFrame;
import javax.swing.JOptionPane;
import java.util.Scanner;

public class LoginGUI extends JFrame {
```

JFrame

>a fundamental class in **Java Swing** for creating and managing windows

```
private void jshowpasswordActionPerformed(java.awt.event.ActionEvent evt) {  
  
    if (jshowpassword.isSelected()) {  
        jLoginPassword.setEchoChar((char) 0);  
    }  
    else {  
        jLoginPassword.setEchoChar(c: '*');  
    }  
  
}
```

Password :

☐ Show Password

The “**Show Password**” feature provides users with flexibility and transparency during login process

NOT showing password

>>>>

Password :

☐ Show Password

Password :

Hello!

☒ Show Password

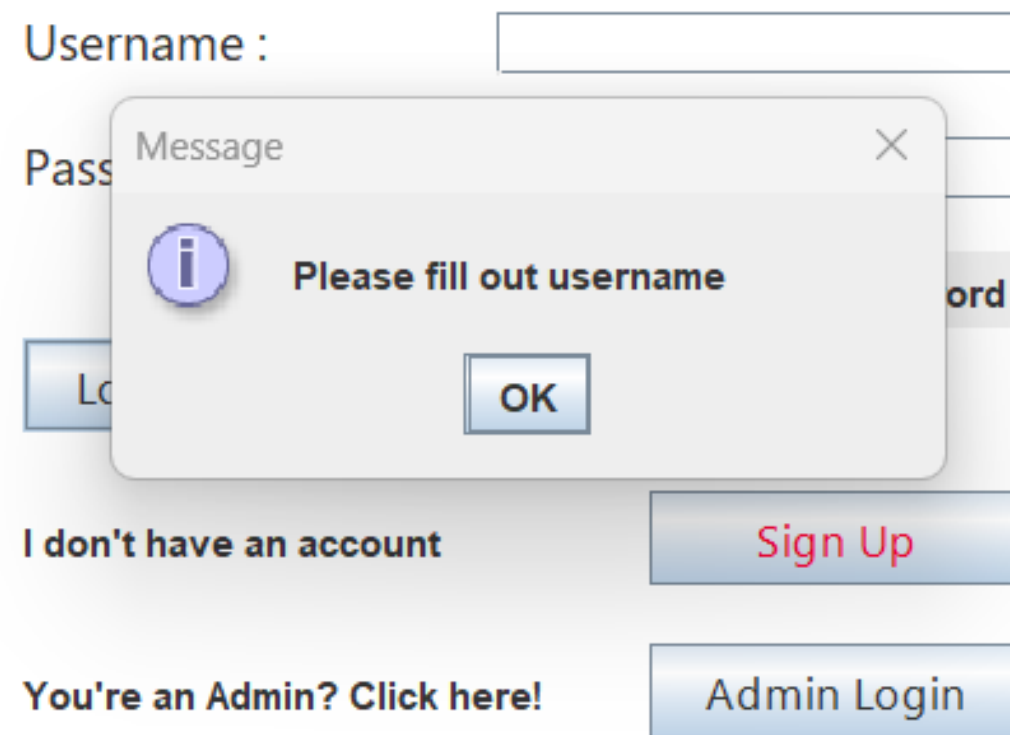
<<<<

Showing password


```
private void jLoginButtonActionPerformed(java.awt.event.ActionEvent evt) {  
  
    if (jLoginUsername.getText().equals("")) {  
        JOptionPane.showMessageDialog(parentComponent: null, message: "Please fill out username");  
    } else if (jLoginPassword.getPassword().length == 0) {  
        JOptionPane.showMessageDialog(parentComponent: null, message: "Please fill out password");  
    } else {  

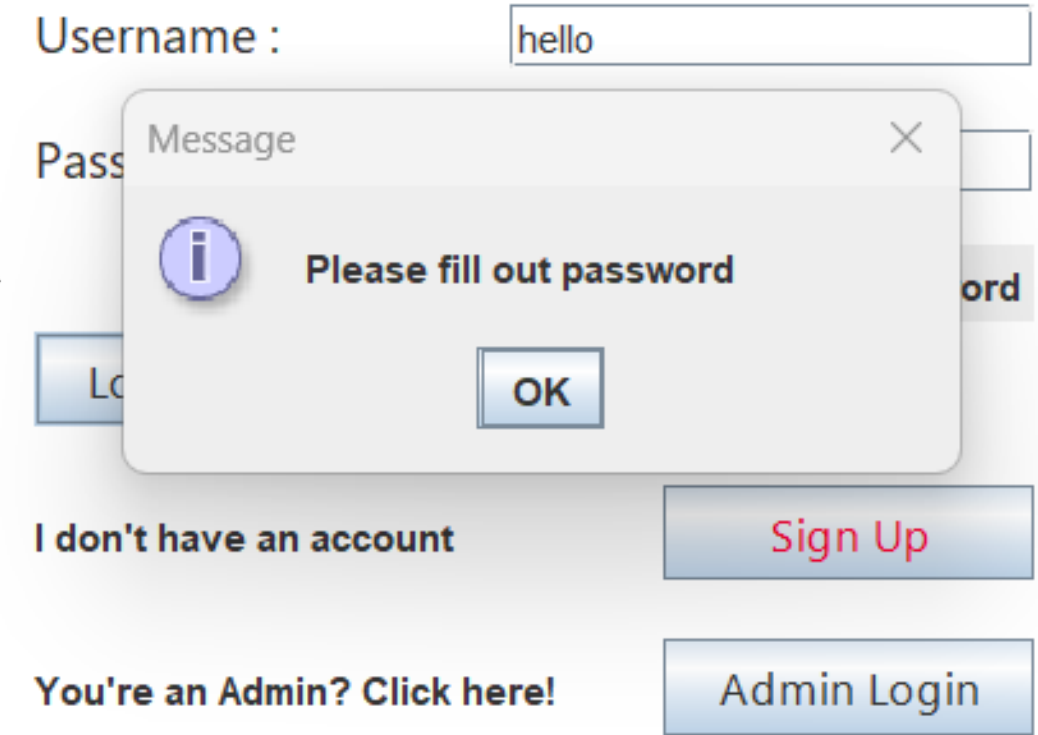
```

If the **username** or **password** field is empty, it will show a message prompting the user to fill out the username and password if they try to login.



JOptionPane

>it's a class that provides a simply way to display dialog boxes for user interaction, such as showing messages.





```
String enteredUsername = jLoginUsername.getText();  
char[] enteredPasswordChars = jLoginPassword.getPassword();  
String enteredPassword = new String(value: enteredPasswordChars);
```

used to retrieve the text and characters entered in the username and password field.

getText() method

used to retrieve the text entered in the username field (**jLoginUsername**) of the login GUI

getPassword() method

used to retrieve the password as an array of characters entered in the password field (**jLoginPassword**)



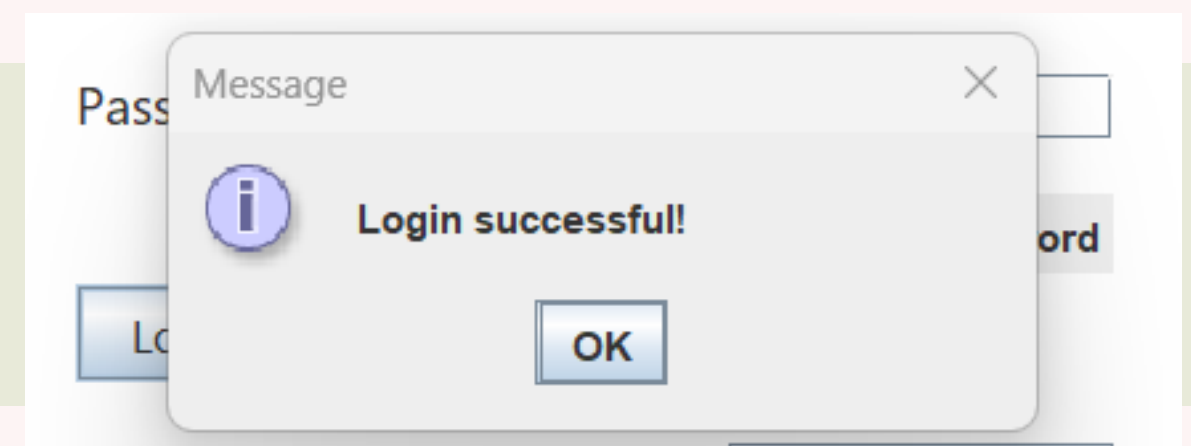
```
if (enteredUsername.equals(anObject: registeredUsername) && enteredPassword.equals(anObject: registeredPassword)) {
```

used to check if the entered **username** and **password** match the registered credentials

```
JOptionPane.showMessageDialog(parentComponent: null, message: "Login successful!");  
dispose();  
  
System.out.println("Welcome, " + enteredUsername + "!");  
  
MainPage proceed = new MainPage();  
proceed.proceedToFeatures();
```

if they are valid, a **JOptionPane** message
will pop out, indicating a successful login.
And the Login GUI will be disposed.

>>



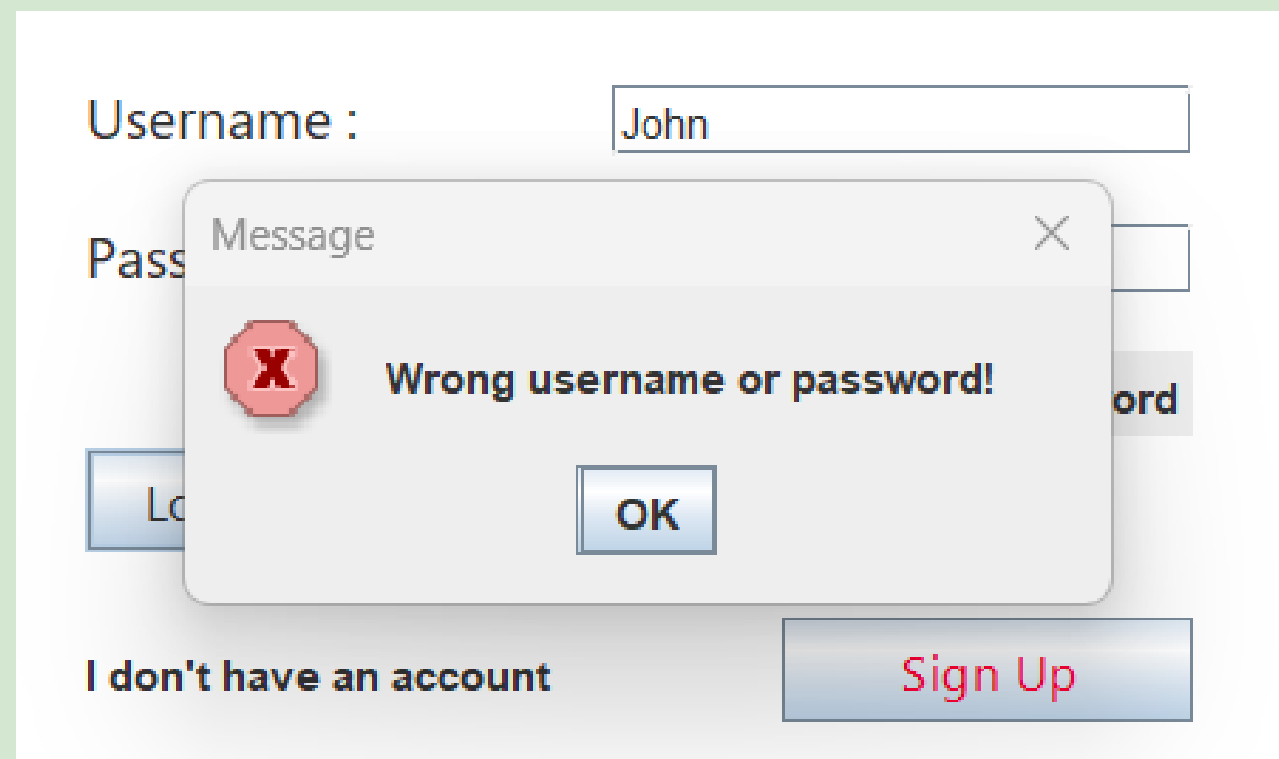
```
Please login to proceed.  
Welcome, John!
```

```
-----  
These are the features available:
```

<<

it will print a welcome message
to the console, and proceeding
to the features in **Main Page**

```
} else {  
    JOptionPane.showMessageDialog(parentComponent: null, message: "Wrong username or password!", title: "Message", messageType: JOptionPane.ERROR_MESSAGE);  
}
```



If the user enter username or password that do not match the registered credentials, a **JOptionPane** with an error message is triggered.

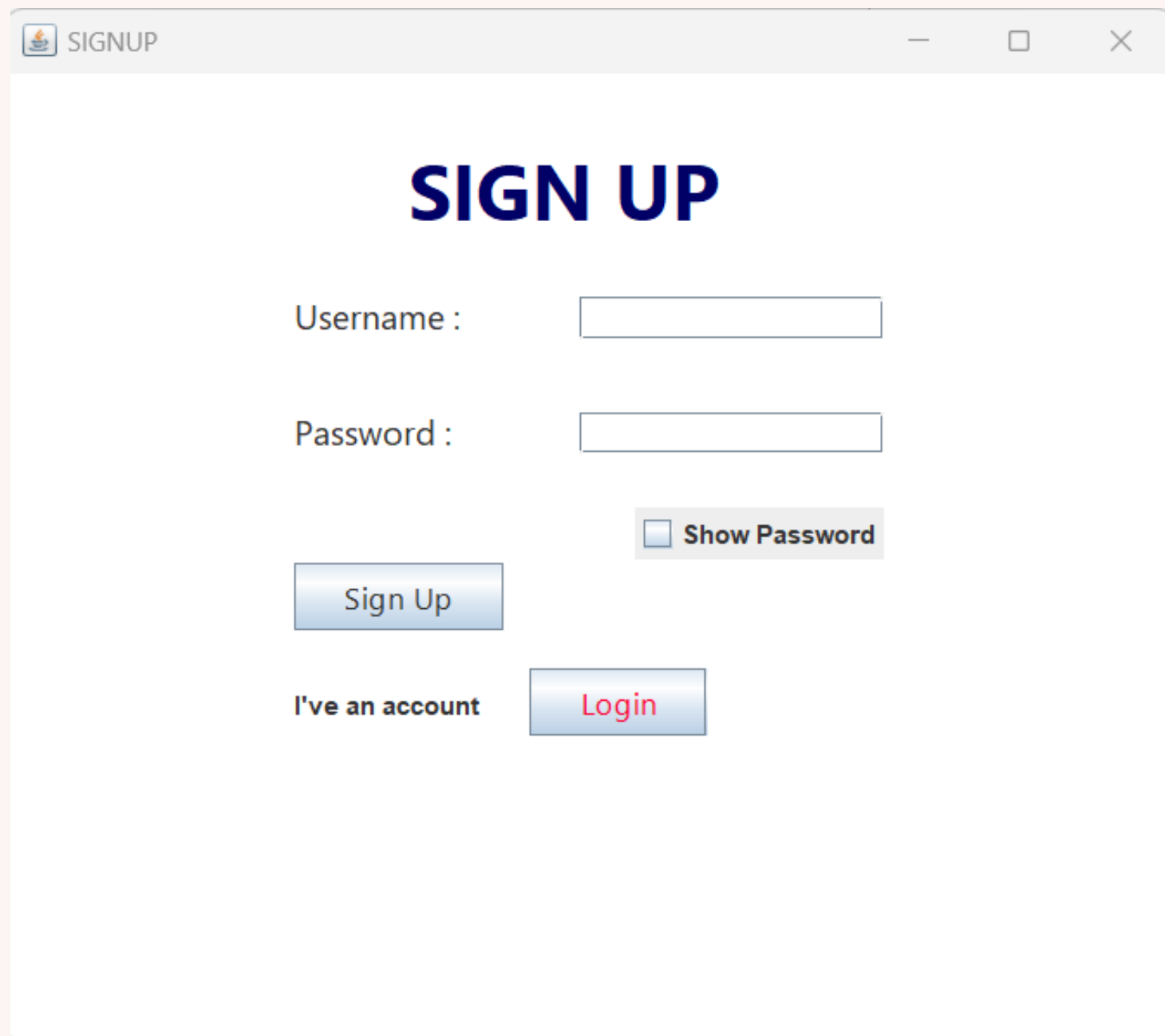
The user can reenter their username and password in order to log in.

Alternatively, if the user does not have an account, they can proceed to **register**.

[I don't have an account](#)

[Sign Up](#)

Sign Up GUI



The screenshot shows a Java Swing window titled "SIGNUP". Inside the window, the text "SIGN UP" is displayed in a large, bold, dark blue font. Below this, there are two text input fields: the first is labeled "Username :" and the second is labeled "Password :". To the right of the password field is a checkbox labeled "Show Password". Below the input fields, there is a blue button labeled "Sign Up". At the bottom left, the text "I've an account" is followed by a blue button labeled "Login" in red text.

```
private void jSignUpButtonActionPerformed(java.awt.event.ActionEvent evt)
{
    SignUpGUI SignUpFrame = new SignUpGUI();
    SignUpFrame.setVisible(b: true);
    SignUpFrame.pack();
    SignUpFrame.setLocationRelativeTo(c: null);
    this.dispose();
}
```

LoginGUI Class

The **Sign Up GUI** will pop out when the user clicks the “**Register**” button in the LoginGUI.

```
private void jSignUpLoginActionPerformed(java.awt.event.ActionEvent evt) {
    LoginGUI LoginFrame = new LoginGUI();
    LoginFrame.setVisible(b: true);
    LoginFrame.pack();
    LoginFrame.setLocationRelativeTo(c: null);
    this.dispose();
}
```

SignUpGUI Class

The “**Login**” button brings the user back to **Login GUI** after they have completed the registration process or if they already have an existing account.

Username :

Password :

User have the flexibility to register with any username of their choice.

```
private void jsignupshowpasswordActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
    if (jsignupshowpassword.isSelected()) {  
        jsignuppassword.setEchoChar ((char) 0);  
    }  
    else {  
        jsignuppassword.setEchoChar (c: '*');  
    }  
}
```

SignUpGUI also provide users with the option to view the entered password

Password :

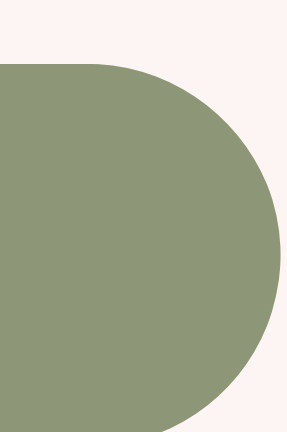
☐ Show Password



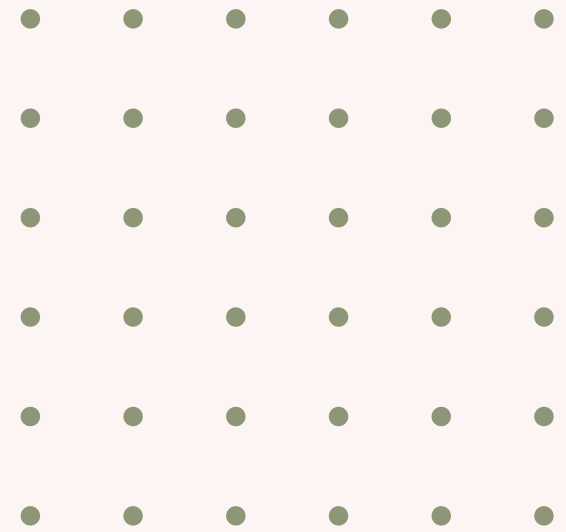
Password :

☒ Show Password

We allowed the users to verify their password before submission.



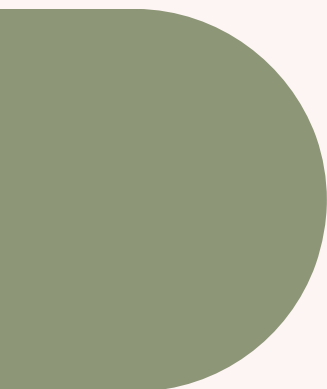
```
private void jSignUpButtonActionPerformed(java.awt.event.ActionEvent evt) {  
  
    String username = jSignUpUsername.getText();  
    String password = new String(value: jsignuppasword.getPassword());  
  
    LoginGUI.setRegisteredUser(username, password);  
}
```



Here, we assign the values entered by the user for the username and password to variables named '**username**' and '**password**' during the registration process

```
LoginGUI.setRegisteredUser(username, password);
```

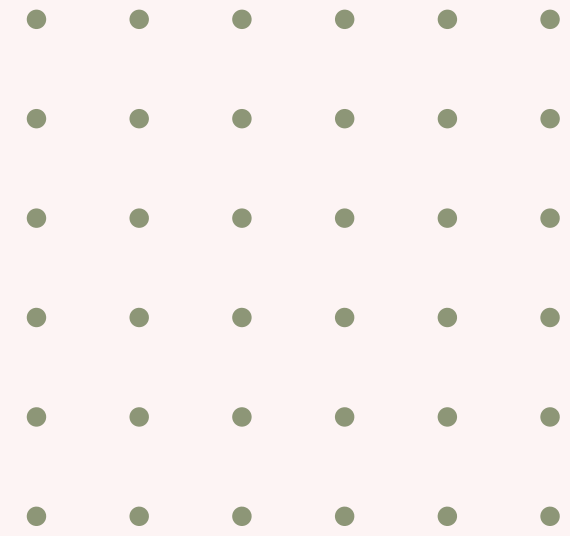
the **username** and **password** are then set in the **LoginGUI** using the **setRegisteredUser()** method



```
public class LoginGUI extends JFrame {

    private static String registeredUsername;
    private static String registeredPassword;

    public static void setRegisteredUser(String username, String password) {
        registeredUsername = username;
        registeredPassword = password;
    }
}
```



After the registration, the entered username and password during login attempts are compared with the credentials that were registered earlier.

```
if (enteredUsername.equals(anObject: registeredUsername) && enteredPassword.equals(anObject: registeredPassword)) {

    JOptionPane.showMessageDialog(parentComponent: null, message: "Login successful!");
    dispose();
}
```

If the entered credentials match the registered ones, a successful login message is displayed in the console.

```

public static void proceedToFeatures() {
    Scanner sc = new Scanner(System.in);
    outerLoop:
    while (true) {
        String featureChoice;
        int featureNumber = 0;

        while (true) {
            System.out.println(x: "\n-----");

            if (isAdmin) {
                System.out.println(x: "These are the features available:");
                for (String feature : features) {
                    System.out.println(x: feature);
                }
            } else {
                System.out.println(x: "These are the features available:");
                System.out.println(x: "2. Information Hub");
                System.out.println(x: "3. Carbon Footprint Calculator");
                System.out.println(x: "4. Community Engagement and Collaboration");
                System.out.println(x: "5. Recent Weather Checker");
                System.out.println(x: "6. Quiz System");
                System.out.println(x: "7. Feedback and Rating System");
            }
            System.out.println(x: "-----");
            System.out.println(x: "Enter 'EXIT' to Logout.");

            System.out.print(s: "Enter the number of your choice: ");

```

while (true)
an infinite loop unless it
meet 'break outerLoop:'

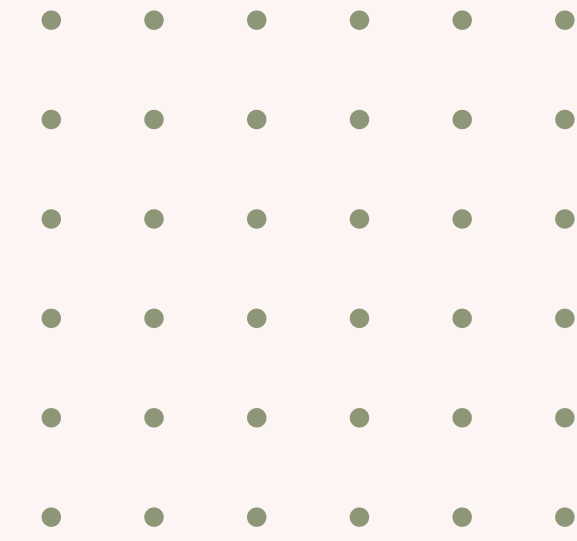
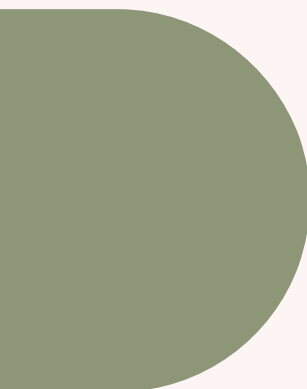
outerLoop
used to exit from the nested
while loop when specific
conditions are met

Output >>>

```

-----
These are the features available:
2. Information Hub
3. Carbon Footprint Calculator
4. Community Engagement and Collaboration
5. Recent Weather Checker
6. Quiz System
7. Feedback and Rating System
-----
Enter 'EXIT' to Logout.
Enter the number of your choice:

```



```
featureChoice = sc.next();

if ("EXIT".equalsIgnoreCase(anotherString: featureChoice)) {
    System.out.println(x: "Exiting the program...");
    break outerLoop;
} else if ("BACK".equalsIgnoreCase(anotherString: featureChoice)) {
    break;
```

equalsIgnoreCase()

used to compare strings without considering the case differences. The condition check if the user's input which stored in **featureChoice** matches 'EXIT'

If the users enter **EXIT**, the 'break outerLoop;' will be executed, breaking all the loops and exit our application

After exploring a feature, users can easily go back to the feature menu by entering **BACK**, because it breaks the loop

Admin Features



■ In addition to the functionality provided to public users, our programme provides additional editing functionality for admin users.

■ Next let's look at the source code to see how we implemented this feature



Since the admin function is nested in the MainPage class, the code will be extracted for the admin function from the MainPage class.

```
private static void adminFeatures() {
    Scanner sc = new Scanner(System.in);
    System.out.println(" ");
    System.out.println("\nAdmin Features:");
    System.out.println("8. Edit or Delete Features");
    System.out.print("Enter the number of your choice: ");

    String userInput = sc.next().toUpperCase();
    if ("EXIT".equals(userInput)) {
        System.out.println("Exiting the program...");
        System.exit( status: 0);
    }

    while (!isValidInteger(userInput)) {
        System.out.println("Invalid input. Please enter a valid integer option.");
        userInput = sc.next().toUpperCase();
        if ("EXIT".equals(userInput)) {
            System.out.println("Exiting the program...");
            System.exit( status: 0);
        }
    }
}
```

When a user is determined to be admin, they are first prompted to enter their choice, which corresponds to the list of administrator functions.

Next, the input is converted to upper case so that it is not case-sensitive.

Then if the user enters "EXIT", the programme exits.


```

int adminChoice = Integer.parseInt(userInput);

if (adminChoice == 8) {
    editOrDeleteFeatures();
} else if (adminChoice >= 2 && adminChoice <= features.length + 1) {
    int featureIndex = adminChoice - 2;
    switch (featureIndex) {
        case 0:
            InformationHub oopinformation = new InformationHub();
            oopinformation.information();
            break;
        case 1:
            CarbonFootprint oopcarbon = new CarbonFootprint();
            oopcarbon.carbon();
            break;
        case 2:
            Community oopcommunity = new Community();
            oopcommunity.comments();
            break;
        case 3:
            Realtime ooprealtime = new Realtime();
            ooprealtime.weather();
            break;
        case 4:
            quiz oopquiz = new quiz();
            oopquiz.test();
            break;
        case 5:
            Feedback oopfeedback = new Feedback();
            oopfeedback.FeedbackandRating();
            break;
        default:
            System.out.println("Invalid choice. Please enter a valid option.");
    }
} else {
    System.out.println("Invalid choice. Please enter a valid option.");
}

```

This code indicates that first the user input is validated to ensure that it is a valid integer option.

If the user selects "8", the editOrDeleteFeatures method is called, allowing the administrator to edit or delete features.

If the user selects any other number, the program performs the appropriate action based on the selection, such as executing the associated method for a different feature.

Overall, the adminFeatures approach provides admin users with an interface and options for managing system functions.

editOrDeleteFeatures method

```
private static void editOrDeleteFeatures() {
    System.out.println("\nCurrent Features:");
    for (String feature : features) {
        System.out.println(feature);
    }

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter the number of the feature to edit or delete: ");
    int userChoice = sc.nextInt();
    sc.nextLine();

    int featureNumber = userChoice - 2;

    if (featureNumber >= 0 && featureNumber < features.length) {
        System.out.print("Enter the new name for the feature: ");
        String newFeatureName = sc.nextLine();

        features[featureNumber] = userChoice + ". " + newFeatureName;
        System.out.println("Feature edited successfully." + "\n" + "Type BACK to return.");
    } else {
        System.out.println("Invalid feature number.");
    }
}
```



The method begins by displaying a list of current system features so that administrators know which features can be edited or deleted.

The scanner is then used to get the number of the feature to be edited or deleted from the user.

Calculates the index of the user-selected feature in the array features (since the array index starts at 0, you need to subtract 2).


```
private static void editOrDeleteFeatures() {
    System.out.println("\nCurrent Features:");
    for (String feature : features) {
        System.out.println(feature);
    }

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter the number of the feature to edit or delete: ");
    int userChoice = sc.nextInt();
    sc.nextLine();

    int featureNumber = userChoice - 2;

    if (featureNumber >= 0 && featureNumber < features.length) {
        System.out.print("Enter the new name for the feature: ");
        String newFeatureName = sc.nextLine();

        features[featureNumber] = userChoice + ". " + newFeatureName;
        System.out.println("Feature edited successfully." + "\n" + "Type BACK to return.");
    } else {
        System.out.println("Invalid feature number.");
    }
}
```

Check that the function number is valid and ensure that the function selected by the user exists.

If the feature number is valid, prompt the administrator to enter a new feature name

Then it will update the corresponding function name in the system function list.

Then the message of successful editing is displayed and the user is prompted to type "BACK" to return to the previous level.

If the function number is invalid, the corresponding error message is displayed.



```
private static String[] features = {  
    "2. Information Hub",  
    "3. Carbon Footprint Calculator",  
    "4. Community Engagement and Collaboration",  
    "5. Recent Weather Checker",  
    "6. Quiz System",  
    "7. Feedback and Rating System"  
};
```

The features array is a class static member variable used to store descriptive information about the various features provided by the system.

The array is designed as an array of strings, where each element is a string containing a feature number and a description of the feature.

```
private static String[] features = {  
    "2. Information Hub",  
    "3. Carbon Footprint Calculator",  
    "4. Community Engagement and Collaboration",  
    "5. Recent Weather Checker",  
    "6. Quiz System",  
    "7. Feedback and Rating System"  
};
```

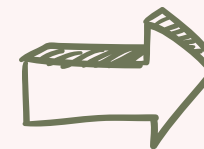
Here, you can see that the first element of this array begins with “2.Information Hub”.

This is because according to the MainPage class, when the user logs in, "1.Login" will be hidden.

So that's why the array index is subtracted by 2

This is because according to the MainPage class, when the user logs in, "1.Login" will be hidden.

So that's why the array index is subtracted by 2.



```
These are the features available:  
2. Information Hub  
3. Carbon Footprint Calculator  
4. Community Engagement and Collaboration  
5. Recent Weather Checker  
6. Quiz System  
7. Feedback and Rating System  
-----  
Enter 'EXIT' to Logout.  
Enter the number of your choice:  
  
Admin Features:  
8. Edit or Delete Features  
Enter the number of your choice:
```

Information Hub

Welcome to the heart of our initiative — the Information Hub. As we embark on a journey to deepen our understanding of climate change, this hub serves as the compass guiding us through a wealth of knowledge, expertly curated to unravel the complexities of our changing planet.



Inside the Information Hub...

Our Information Hub is thoughtfully categorized into distinct topics, providing a structured approach to understanding and addressing climate change.

Explore the following key categories:

- 1) Climate Change Basics**
- 2) Education and Awareness**
- 3) Mitigation Strategies**
- 4) Adaptation Measures**
- 5) Impact Reduction Techniques**



Climate Change Basics

Within the "Climate Change Basics" section, we meticulously cover fundamental concepts essential for building a solid understanding of this critical subject. In this topic, it contains...

- 1) Definition of global warming**
- 2) How it contributes to climate change.**
- 3) What are greenhouse gases**
- 4) Climate vs Weather**



Education and Awareness

Within the dedicated section of "Education and Awareness," our Information Hub aims to shed light on pivotal aspects that contribute to a well-informed and environmentally conscious society.

Explore the following key components:

- 1) Importance of Climate Education**
- 2) Educational Programs and Initiatives**
- 3) Public Awareness Campaigns**



Mitigation Strategies

Dive into the realm of proactive solutions with our "Mitigation Strategies" section, where we outline key approaches to address and alleviate the impacts of climate change. Explore the following essential components:

- 1) Definition of mitigation of climate change**
- 2) Renewable Energy**
- 3) Energy Efficiency**
- 4) Sustainable Transportation**



Adaptation Measures

Explore the realm of resilience and preparedness in our "Adaptation Measures" section, where we outline essential strategies to navigate and mitigate the impacts of climate change. Uncover the following key components:

- 1) Climate-Resilient Agriculture**
- 2) Urban Planning for Climate Resilience**
- 3) Water Management and Adaptation**



Impact Reduction Techniques

Embark on a journey of minimizing environmental impact with our "Impact Reduction Techniques" section, where we explore strategies aimed at fostering sustainability and conservation.

Uncover the following key components:

- 1) Biodiversity Conservation**
- 2) Sustainable Practices**
- 3) Waste Management**



Behind the scenes

```
1 package oop.group.project;
2
3 import java.util.Scanner;
4
5 public class InformationHub {
6     public void information() {
7         Scanner sc = new Scanner(System.in);
```

import java.util.Scanner

This line imports the Scanner class from the java.util package. The Scanner class is used for taking input from the user.

public class InformationHub

This declares a class named InformationHub. The class is declared as public, meaning it can be accessed from other classes.

public void information()

This declares a method named information within the InformationHub class. The method is declared as public and has a return type of void, indicating that it doesn't return any value. It doesn't take any parameters.

Scanner sc = new Scanner(System.in)

Inside the information method, a new Scanner object named sc is created. This object is used to read input from the standard input (keyboard), represented by System.in.


```
while(true){
System.out.println(x: "\nWelcome to Information Hub!");
System.out.println(x: "");
System.out.println(x: ""
                        The Information Hub feature serves as a central hub for information related to
                        climate change, offering users a one-stop destination for exploring various topics, accessing
                        resources, and the latest information on climate change.
System.out.println(x: "");
System.out.println(x: "In this Information Hub, you can explore and learn about various aspects of climate change.");
System.out.println(x: "");
System.out.println(x: "-----");
System.out.println(x: "1) Climate Change Basics ");
System.out.println(x: "2) Education and Awareness ");
System.out.println(x: "3) Mitigation Strategies ");
System.out.println(x: "4) Adaptation Measures ");
System.out.println(x: "5) Impact Reduction Techniques ");
System.out.println(x: "-----");
System.out.println(x: "");
System.out.println(x: "Please enter a topic number to search: ");
System.out.println(x: "Enter 'BACK' to go back to the Main Page.");

String userInput = sc.next();
```

```
while(true)
```

This starts an infinite loop. The while (true) construct ensures that the following block of code will keep running until explicitly broken out of.

```
System.out.println("Please enter a topic number to  
search")
```

```
System.out.println("Enter 'BACK' to go back to the  
Main Page")
```

Prompts the user to enter a topic number for more information. It also informs the user that they can enter 'BACK' to return to the main page.

```
String userInput = sc.next();
```

Reads the user's input using the previously declared Scanner object sc.

```
if ("BACK".equalsIgnoreCase (anotherString: userInput)) {  
    break;  
}
```

equalsIgnoreCase:

This method is used to compare two strings, ignoring their case. In this case, it checks if the user's input is equal to "BACK," regardless of whether the letters are in uppercase or lowercase.

break:

If the condition is true (if the user entered "BACK"), the break statement is executed. This terminates the while loop, and the program will continue with the code that follows the loop.


```
int topicNumber = Integer.parseInt(s: userInput);
```

```
switch(topicNumber) {
```

```
    case 1:
```

```
        handleClimateChangeBasics(a: sc);
```

```
        break;
```

```
    case 2:
```

```
        handleEducationAndAwareness(a: sc);
```

```
        break;
```

```
    case 3:
```

```
        handleMitigationStrategies(a: sc);
```

```
        break;
```

```
    case 4:
```

```
        handleAdaptationMeasures(a: sc);
```

```
        break;
```

```
    case 5:
```

```
        handleImpactReductionTechniques(a: sc);
```

```
        break;
```

```
    // Add cases for other topics if needed
```

```
    default:
```

```
        System.out.println(x: "Invalid input. Please enter a valid topic number or 'EXIT'.");
```

```
    }
```

```
}
```

int topicNumber = Integer.parseInt(userInput);

Converts the user's input (which is assumed to be a string representing a number) into an integer using Integer.parseInt(). This assumes the user has entered a valid numeric topic number.

switch(topicNumber)

Initiates a switch statement based on the value of topicNumber. It checks the value against different cases.

```
int topicNumber = Integer.parseInt(s: userInput);
```

```
switch(topicNumber) {  
    case 1:  
        handleClimateChangeBasics(a: sc);  
        break;  
    case 2:  
        handleEducationAndAwareness(a: sc);  
        break;  
    case 3:  
        handleMitigationStrategies(a: sc);  
        break;  
    case 4:  
        handleAdaptationMeasures(a: sc);  
        break;  
    case 5:  
        handleImpactReductionTechniques(a: sc);  
        break;  
    // Add cases for other topics if needed  
    default:  
        System.out.println(x: "Invalid input. Please enter a valid topic number or 'EXIT'.");  
}  
}
```

case 1:

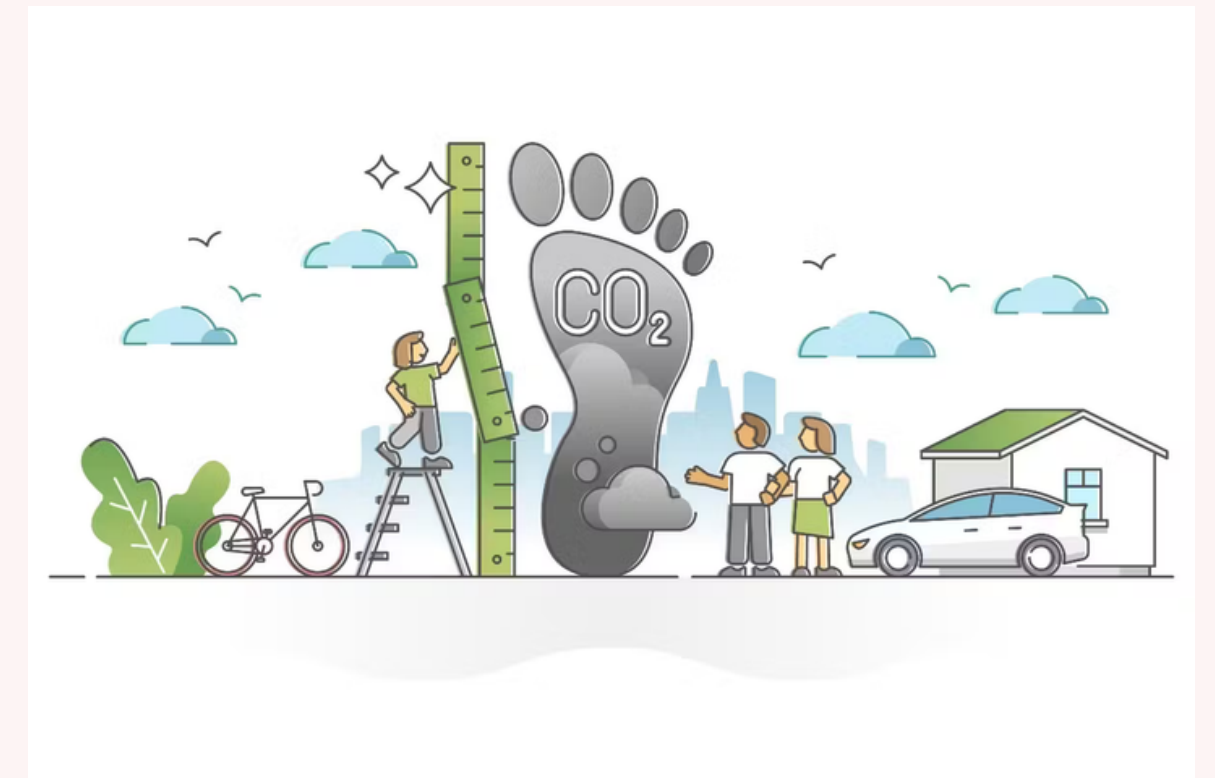
Each case represents a specific topic number. If the user entered 1, it calls the `handleClimateChangeBasics` method and then break out of the switch statement.

default:

This is the default case that gets executed if the user's input does not match any of the specified cases. It prints an error message indicating that the input is invalid and prompts the user to enter a valid topic number or 'BACK'.

Carbon Footprint Calculator

- Introducing one of our important features which users can learn how they can contribute better and leave a positive impact to the climate.
- With this, users will be aware and informed of how many initiatives they have taken which turn into useful data showing how much carbon footprints they have reduced in a day.



Behind the scenes

```
1 package oop.group.project;
2 import java.util.Scanner;
3
4 public class CarbonFootprint {
5
6
7     public static void carbon() {
8         while (true) {
9
10             Scanner sc = new Scanner(System.in);
```

import java.util.Scanner; This import Scanner class from the java.util package. This class is imported to take in user input

public class CarbonFootprint A class name CarbonFootprint is declared. It is public class meaning it can be accessed from other class

public static void carbon() A method name carbon is declared within the CarbonFootprint class. This method is public and has return type void which does not return any value

while (true) An infinite loop for the carbon method that will always be true unless stop by external intervention

Scanner sc = new Scanner(System.in) A new Scanner object is created inside the carbon method. This read input from user input by keyboard, represented by System.in

```
// Introduction and advantages
System.out.println(x: "\nWhat is carbon footprint?\nCarbon footprint is the sum of greenhouse gases emitted such as ca
System.out.println(x: "");
System.out.println(x: "Advantage of walking ro reduce carbon footprint:");
System.out.println(x: "Reduce air pollution to reduce the effects of climate change\nImprove health especially minimis
System.out.println(x: "");

// Basic understanding of how the calculation works
System.out.println(x: "Note: How carbon footprints reduced by walking is calculated:");
System.out.println(x: "1 footstep is equivalent to 0.0007576km (estimated)");
System.out.println(x: "The more the footsteps count, the more the distance, and the more the carbon footprints are red
System.out.println(x: "");
```

- System prints out the definition of carbon footprint and advantages of walking to reduce carbon footprint
- System prints out some basic understanding on how this feature works


```
// Calculation
System.out.println(x: "The calculation of the amount of carbon footprint reduced by walking.");

System.out.println(x: "Enter your footsteps of the day");

System.out.println(x: "Enter 'BACK' to go back to the main page");
String userInput = sc.next();

if ("BACK".equalsIgnoreCase(anotherString: userInput)) {
    break;
}

int footsteps = Integer.parseInt(s: userInput);
```

System prints out both sentences that prompt the user to enter their footsteps of the day and enter 'BACK' to go back to main page if user don't want to continue

String userInput = sc.next(); Reads the user's input assigned for the variable declared as userInput as String datatype

if ("BACK".equalsIgnoreCase(userInput)) If user's input is back, ignoring the case, the while loop stopped and bring user back to the main page

int footsteps = Integer.parseInt(userInput); This allow change of datatype to integer as the scanner previously only reads String datatype, therefore user can enter number of footsteps


```
System.out.println(x: "");  
  
double distanceTravelled = footsteps / 1320.0;  
System.out.println(x: "-----");  
System.out.println("Your distance travelled: " + distanceTravelled + "km");  
  
System.out.println(x: "");
```

- **double distanceTravelled = footsteps / 1320.0;** Basic calculation of distance travelled where footsteps taken from user's input is divided by 1320.0, where the output is declared as double datatype with the variable name distanceTravelled
- System prints out the the sentence showing how much distance travelled in km with the output assigned to the variable distanceTravelled from the calculation mentioned above

```
// the emissions conversion factors per kilometer for car generally
double averageCarWithPetrol = 0.17431;

double calculation1 = averageCarWithPetrol * distanceTravelled;

System.out.println("Carbon footprint reduced you have contributed for the day: " + calculation1 + " Kg CO2e");
System.out.println(x: "-----");
```

■ **double averageCarWithPetrol = 0.17431;** The decimal number 0.17431 (Tayfusion, 2021) is assigned to the variable name averageCarWithPetrol as double datatype

■ **double calculation1 = averageCarWithPetrol * distancedTravelled;** (Tayfusion, 2021) Math calculation where output of the first variable is multiplied with the output of second variable to give the output of variable name calculation1 as double datatype

(Tayfusion, 2021)

Community Engagement and Collaboration

- This is where we embark on a journey of collective action and shared responsibility.
- Do you have any questions or need any guidances? Ask our professionals! Let's join forces and support one another in fighting climate change, and making the world cleaner and more sustainable!



Behind the scenes

```
while (true){  
    System.out.println(x: "\nWelcome to the comment/forum section!");  
    System.out.println(x: "Here, you can converse with other members and asks questions from experts.");  
    System.out.println(x: "Type 'Continue' to continue into the comment/forum section.");  
    System.out.println(x: "Type 'Back' to go back to Main Page.");  
    Scanner cont = new Scanner(source: System.in);  
    String x = cont.nextLine();  
}
```

- **while(true)** is an infinite loop as it is always true (only stopped by external intervention)
- the system prints out the introduction by using the **System.out.println()** method
- **new Scanner** created an object of scanner class to take in user input with **System.in**
- **nextLine()** takes in user input and assigns it to the **String x**.

```
List<String> comments = new ArrayList<String>();  
if (x.equalsIgnoreCase(anotherString: "Continue")){  
  
System.out.println(x: "\nForum:");  
    System.out.println("John Doe (expert) commented:\n "  
        + "Hi everybody! My name is John Doe and I will be answering any of your questions related in this field.");  
}
```

■ **List<String> comments = new ArrayList<String>()** creates an object of ArrayList of Strings

■ **if (x.equalsIgnoreCase("Continue"))** if the user input x, is equal to “continue”, while ignoring the case, the system prints out the forum page as well as a comment from John Doe saying “Hi everybody! My name is John Doe and I will be answering any of your questions related in this field.”


```
while (true){  
  
    System.out.print(s: "Type 'Back' to go back.\nEnter Your Comment:");  
    String comment = cont.nextLine();  
  
    if ("back".equalsIgnoreCase(anotherString: comment)){  
  
        break;  
    }  
}
```

- **while(true)** is an infinite loop as it is always true
- the system then prints out "Type 'Back' to go back.\nEnter Your Comment:" with the **print function**. **\n** used to create a newline. The system is asking the user to input their comment
- the system then takes in user comment with **nextLine()** and assigns it to **String comment**.
- **if ("back".equalsIgnoreCase(comment))** if user input is equals to “back” while ignoring the case, the while loop is stopped with **break** and sends the user back

```
private static void displayComments(List<String> comments) {  
    System.out.println(x: "\nYour Comments:");  
    for (String comment : comments) {  
        System.out.println("You commented: " + comment);  
    }  
}
```

private static void displayComments(List<String> comments), this creates a method named 'displayComments'. it can only be accessed within the same class where it is defined using '**private**'. (**List<String> comments**) this indicates that it can only take a single parameter '**comments**' of type **List<String>**

for (String comment : comments) this means that for every element in the ArrayList **comments** the system will print out "**You commented:**" + **comment** with the **print** command

```
System.out.println("John Doe (expert) commented:\n "  
    + "Hi everybody! My name is John Doe and I will be answering any of your questions related in this field.");  
comments.add(e: comment); //add into array list  
displayComments(comments); //displays the contents
```

- The system prints out John Doe's comment with the **System.out.println() method**
- **comments.add(comment)** this adds the user's input comment into the **ArrayList comments**
- **displayComments(comments)** this calls on the method called 'displayComments' and takes in comments as its argument and it will print out the user's comments

```
else if (x.equalsIgnoreCase(anotherString: "Back")){  
    break;  
}  
else {  
    System.out.println(x: "You have entered an invalid choice. Sending user back to main menu.");  
    break;  
}
```

- **else if (x.equalsIgnoreCase("Back"))** if the user's x input is equal to "Back" it sends the user back to the main menu with the **break** command (exit the comment section).
- **else** if the user inputs any invalid options then, it will print "You have entered an invalid choice. Sending user back to main menu." with the **print** command and sends the user back to main menu with the **break** command.

Recent Weather Checker

Obviously, this is a program about recent weather queries. In this program you can check the detailed weather conditions for the last seven days, such as: temperature, body temperature, air quality, wind speed, humidity, visibility, barometric pressure, dew point, date .

Next let's see how this program is written.



Behind the scenes

```
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
9, 68, 9, 1009, 17, 17}, {23, 24, 46, 8, 66, 10, 1008, 16, 16}, {22, 23, 44, 7, 64, 11, 1007, 15, 15}, {21, 22, 42, 6, 62, 12, 1006, 14, 14}, {20, 21, 40, 5, 60, 13, 1
```

- I first created a multidimensional array to store the weather information for the last seven days for each location.

- They represent: temperature, body temperature, air quality, wind speed, humidity, visibility, barometric pressure, dew point, and date.

```
, "Melaka(马六甲)", "Negeri Sembilan(森美兰)", "Pahang(彭亨)", "Perak(霹靂)", "Johor(柔佛)", "Sabah(沙巴)", "Sarawak(沙撈越)", "Selangor(雪兰莪)", "Pulau Pinang(槟城)",  
"Air quality(空气质量)", "Air velocity(风速)", "Humidity level(湿度)", "Increased visibility(能见度)", "Pneumatic(气压)", "Dew point(露点)", "Dates(日期)");
```

```
static String[] regions = {"Kedah(吉打)"
```

```
static String[] attributes = {"Temperat
```

■ Then these two arrays regions and attributes were created to provide the names of the regions and weather attributes to make the application more readable and user-friendly.

```
public static void weather() {  
    try {  
        System.setOut(new java.io.PrintStream(System.out, autoFlush: true, encoding: "UTF-8"));  
    } catch (UnsupportedEncodingException e) {  
        e.printStackTrace();  
    }  
}
```

As you can see in the previous content, the design of the region name and attribute name in addition to the English language is also used in Chinese, so I have to think of a way to make the Chinese can be output out normally.

As you can see, in order to make the Chinese language and symbols can be printed out correctly, I use utf-8 character encoding, and in order to ensure the robustness and reliability of the program, I also use try-catch, because at the time of coding I can not be sure of the external environment on the character encoding support.

```
while(true){

    System.out.println("\nWelcome to the Weather Checker program!/欢迎使用天气查询程序! ");
    System.out.println("Enter BACK to go back to Main Page");
    System.out.println("Enter PLAY to enter the program");

    String feedback = sc.nextLine();
    if ("PLAY".equalsIgnoreCase(feedback)) {
        selectRegion();
    }
    if ("BACK".equalsIgnoreCase(feedback)) {
        break;
    }

}
```

- This code creates a simple interaction loop. The user is prompted to enter "PLAY" or "BACK";
- if "PLAY" is entered, the selectRegion() method is called and the next interaction stage is entered;
- if "BACK" is entered, the loop is jumped out of and the program ends.

```
while(true){

    System.out.println("\nWelcome to the Weather Checker program!//欢迎使用天气查询程序! ");
    System.out.println("Enter BACK to go back to Main Page");
    System.out.println("Enter PLAY to enter the program");

    String feedback = sc.nextLine();
    if ("PLAY".equalsIgnoreCase(feedback)) {
        selectRegion();
    }
    if ("BACK".equalsIgnoreCase(feedback)) {
        break;
    }

}
```

- If the user enters something else, the loop will continue, redisplaying the welcome message and selection prompt, waiting for user input.
- My design allows the user to control the flow of the programme by entering different commands.


```

public static void selectRegion() {

    System.out.println("\nPlease select a region (enter a number)/请选择一个地区（输入数字）：");

    for (int i = 0; i < regions.length; i++) {
        System.out.println((i + 1) + "： " + regions[i]);
    }

    System.out.println("0: Return to previous level/返回上一级");

    int input = sc.nextInt();
    sc.nextLine();

    if (input < 0 || input > regions.length) {

        System.out.println("Input error, please re-enter!/输入错误，请重新输入！");

        selectRegion();
    } else if (input == 0) {

        System.out.println("Returned to higher level!/已返回上一级！");

        return;
    } else {

        selectDate( regionIndex: input - 1);
    }
}

```

This code first prints out all the regions for the user to choose from and then takes a number from the user input.

If the input is not legal, the user is prompted to re-enter it. If the number entered by the user is 0, indicating a return to the previous level, the method ends.

```

public static void selectRegion() {

    System.out.println("\nPlease select a region (enter a number)/请选择一个地区（输入数字）：");

    for (int i = 0; i < regions.length; i++) {
        System.out.println((i + 1) + ": " + regions[i]);
    }

    System.out.println("0: Return to previous level/返回上一级");

    int input = sc.nextInt();
    sc.nextLine();

    if (input < 0 || input > regions.length) {

        System.out.println("Input error, please re-enter!/输入错误，请重新输入！");

        selectRegion();
    } else if (input == 0) {

        System.out.println("Returned to higher level!/已返回上一级！");

        return;
    } else {

        selectDate( regionIndex: input - 1);
    }
}

```

If the number entered is within the legal range, the selectDate method is called, which goes to the stage of selecting a date and passes the index of the region selected by the user.

```

public static void selectDate(int regionIndex) {

    System.out.println("\nPlease select a date (enter a number)/请选择一个日期（输入数字）：");

    for (int i = 0; i < 7; i++) {
        System.out.println((i + 1) + ": " + "December " + weatherData[regionIndex][i][8] + " / " + "12月" + weatherData[regionIndex][i][8] + "日")
    }

    System.out.println("0: Return to previous level/返回上一级");

    int input = sc.nextInt();

    if (input < 0 || input > 7) {

        System.out.println("Input error, please re-enter!/输入错误，请重新输入！");

        selectDate(regionIndex);
    } else if (input == 0) {

        System.out.println("Returned to higher level!/已返回上一级！");

        selectRegion();
    } else {

        showWeather(regionIndex, dateIndex: input - 1);
    }
}

```

This method first prints out all the dates for the user to choose from and then takes a number from the user input.

If the input is not legal, the user is prompted to re-enter it.

If the number entered by the user is 0, indicating a return to the previous level, the method ends.

```

public static void selectDate(int regionIndex) {

    System.out.println("\nPlease select a date (enter a number)/请选择一个日期（输入数字）：");

    for (int i = 0; i < 7; i++) {
        System.out.println((i + 1) + ": " + "December " + weatherData[regionIndex][i][8] + " / " + "12月" + weatherData[regionIndex][i][8] + "日")
    }

    System.out.println("0: Return to previous level/返回上一级");

    int input = sc.nextInt();

    if (input < 0 || input > 7) {

        System.out.println("Input error, please re-enter!/输入错误，请重新输入！");

        selectDate(regionIndex);
    } else if (input == 0) {

        System.out.println("Returned to higher level!/已返回上一级！");

        selectRegion();
    } else {

        showWeather(regionIndex, dateIndex: input - 1);
    }
}

```


 If the number entered is within the legal range, the showWeather method is called, moving to the stage of displaying the weather information and passing the index of the date selected by the user.

```

public static void showWeather(int regionIndex, int dateIndex) {

    System.out.println("\n以下是" + regions[regionIndex] + "在12月" + weatherData[regionIndex][dateIndex][8] + "日的天气信息: ");
    System.out.println("Here's the weather in " + regions[regionIndex] + "on December " + weatherData[regionIndex][dateIndex][8] + ":");

    for (int i = 0; i < 8; i++) {
        System.out.println(attributes[i] + ": " + weatherData[regionIndex][dateIndex][i] + getUnit(i));
    }

    System.out.println("0: Return to previous level/返回上一级");

    int input = sc.nextInt();

    if (input == 0) {
        System.out.println("Returned to higher level!/已返回上一级! ");
        selectDate(regionIndex);
    } else {
        System.out.println("Input error, please re-enter!/输入错误, 请重新输入! ");
        showWeather(regionIndex, dateIndex);
    }
}

```

This method first prints the header, showing the selected region and date, then iterates through the array of weather attributes and prints the information for each attribute.

Next, it prints the option to return to the previous level, reading the user's input.


```
public static void showWeather(int regionIndex, int dateIndex) {

    System.out.println("\n以下是" + regions[regionIndex] + "在12月" + weatherData[regionIndex][dateIndex][8] + "日的天气信息: ");
    System.out.println("Here's the weather in " + regions[regionIndex] + "on December " + weatherData[regionIndex][dateIndex][8] + ":");

    for (int i = 0; i < 8; i++) {
        System.out.println(attributes[i] + ": " + weatherData[regionIndex][dateIndex][i] + getUnit(i));
    }

    System.out.println("0: Return to previous level/返回上一级");

    int input = sc.nextInt();

    if (input == 0) {
        System.out.println("Returned to higher level!/已返回上一级! ");

        selectDate(regionIndex);
    } else {
        System.out.println("Input error, please re-enter!/输入错误, 请重新输入! ");

        showWeather(regionIndex, dateIndex);
    }
}
```

If the number entered by the user is 0, indicating a return to the previous level, the selectDate method is called. Next, it prints the option to return to the previous level, reading the user's input.

If the number entered is not legal, it prompts the user to re-enter it and calls itself recursively.


```
public static String getUnit(int attributeIndex) {  
    switch (attributeIndex) {  
        case 0:  
        case 1:  
        case 7:  
            return "°C";  
        case 3:  
            return "km/h";  
        case 4:  
            return "%";  
        case 5:  
            return "km";  
        case 6:  
            return " hPa";  
        default:  
            return "";  
    }  
}
```

This method uses a switch statement to determine which property's unit should be returned based on the property index of the input. Different property indices correspond to different units, such as °C for temperature, km/h for wind speed, % for humidity, and so on.

If the input index is not within the known range, an empty string is returned. This way, when displaying weather information, calling this method will give you the units of the corresponding attribute.

Quiz System

- This is where we will embark on a journey of discovery and learning.
- Ready to challenge your newly gained knowledge? Our quiz system is designed to engage minds, promote interactive learning, and provide a platform for testing and expanding your understanding on climate change.



Behind the scenes

```
String [] questions =
{"Human activities, such as burning fossil fuels, contribute to the release of greenhouse gases into the atmosphere.",
 "The natural greenhouse effect is necessary for maintaining a habitable temperature on Earth.",
 "Climate refers to short-term atmospheric conditions, while weather involves long-term patterns.",
 "The primary goal of climate change mitigation is to increase the emission of greenhouse gases into the atmosphere.",
 "Sustainable transportation aims to rely on depleting natural resources like coal, oil, and gas.",
 "What are the primary greenhouse gases responsible for global warming?\nA) Oxygen\nB) Nitrogen\nC) Carbon Dioxide (CO2), Methane (CH4)",
 "What is the main goal of climate change education?\nA) Promoting deforestation\nB) Fostering sustainable practices\nC) Ignoring global",
 "Which organization launched the UNESCO Climate Change Education for Sustainable Development program?\nA) NASA\nB) Citizens' Climate",
 "What is the primary goal of mitigating climate change?\nA) Increasing greenhouse gas emissions\nB) Adapting to climate change\nC) Le",
 "What is a crucial component of climate-resilient agriculture (CRA)?\nA) Depleting natural resources\nB) Promoting hunger and poverty"}
};

String [] answers = {"true", "true", "false", "false", "false", "C", "B", "C", "C", "C"};
Scanner scanner = new Scanner(System.in); // Scanner for start stop
```

■ **String []** is used to create an array of strings for the questions and answers with the answer and question index corresponding to each other

■ The **scanner class** is used to create a **new scanner object** and to take in input from the user. Hence, the use of **(System.in)**

```
while (true) {  
    System.out.println(x: "\nAre you ready for the quiz?");  
    System.out.println(x: "Type 'Continue' to start the quiz.");  
    System.out.println(x: "Type 'Back' to go back to the Main Page.");  
    String x = scanner.next();  
    int points = 0;  
}
```

- **while(true)** is an infinite loop as it is always true
- **scanner.next()** takes in the user input and assigns it to the variable x of data type String
- **int points = 0** zero is assigned to 'points' of integer data type. This sets the default/starting point to zero, creating a point system for the quiz.

```
if (x.equalsIgnoreCase(anotherString: "continue")){  
for(int i = 0; i<10; i++) {  
    System.out.println("\nQuestion:\n" + questions[i]);  
}
```

■ **if (x.equalsIgnoreCase("continue"))** this checks whether the contents of the String x is equal to "continue" while ignoring case. If it is equal to "continue" it will proceed to the **for loop**

■ **for (int i=0; i<10; i++)** this creates a for loop with the variable 'i' starting at 0 with the condition that 'i' is less than 10. After every loop 'i' is added with 1. This means it is being looped 10 times.

■ **System.out.println("\nQuestion:\n" + questions[i])** this command is being looped and is printing the questions with index [i]


```
if (i>=5 && i<10){  
    System.out.print(s: "Your answer:");  
    String uranswer = scanner.next().toUpperCase(); // takes input first before proceeding to next code  
  
    if (uranswer.equals(answers[i])) {  
        System.out.println(x: "You are correct!");  
        points++;  
    }  
    else {  
        System.out.println(x: "You are wrong!");  
    }  
}
```

■ **if (i>=5 && i<10)** this checks the question and in this case is MCQ

■ it then prints **“Your answer:”** asking the user to input their answer in the **scanner.next().toUpperCase()** and automatically changing the user’s input to uppercase

■ the **if** and **else** statement is then used to check if the user’s answer is equal to the correct answer with the **equals()** method. If it is equal, it prints **“You are correct”** and the points increment by 1 and if it is not equal, it prints **“You are wrong”**


```
else {  
    System.out.println(x: "True or False?");  
    String uranswer = scanner.next().toLowerCase();  
    if (uranswer.equals(answers[i])) {  
        System.out.println(x: "You are correct!");  
        points++;  
    }  
    else {  
        System.out.println(x: "You are wrong!");  
    }  
}
```

- **else** checks the question and for this case it is True or False
- it then prints **System.out.println("True or False?")** and takes in the user input from **scanner.next().toLowerCase()** and changes the user's input to lowercase
- the **if** and **else** statement is then used to check if the user's answer is equal to the correct answer with the **equals()** method. If it is equal, it prints "You are correct" and the points increment by 1 and if it is not equal, it prints "You are wrong"

```
        System.out.println("\nCongratulations! You scored: " + points + "/10");
        System.out.println(x: "Type 'BACK' to go back to the main page");
        String y = scanner.next();

        if (y.equalsIgnoreCase(anotherString: "BACK")){
            break;// back command
        }
        else {
            System.out.println(x: "You entered an invalid choice. Sending user back.");
            break;// back command
        }
    }
}
```

At the end of the **for loop**, **System.out.println("\nCongratulations! You scored: " + points + "/10")** is printed along with the final points based on the questions answered correctly

System.out.println("Type 'BACK' to go back to the main page") is printed and the **scanner** takes in user input and assigns it to **y**

if (y.equalsIgnoreCase("BACK")) if the user input is equal to “BACK”, ignoring the case, it **breaks the while loop**. **else** it will print **System.out.println("You entered an invalid choice. Sending user back.")** and breaking the loop with **break**;

```
else if ("BACK".equalsIgnoreCase(anotherString: x)){  
    break;  
}  
else {  
    System.out.println(x: "You entered an invalid choice. Sending user back.");  
    break;// back command  
}
```

- **else if ("BACK".equalsIgnoreCase(x))** this checks if the user's **input x** is equal to "BACK". if it is equal it will break the while loop with the **break;** command without completing the quiz (in the earlier slide if user input is "Continue" it will continue to the quiz)
- **else** this is if the user inputs any other values, the system will print **System.out.println("You entered an invalid choice. Sending user back.")** and executes the **break;** command, sending the user back to the main page

Feedback and Rating System

- This feature allows the users to type in their feedback on using the application and rate the application on a range of scales.
- Therefore, we can improve further on our application based on the feedbacks.



Behind the scene

```
package oop.group.project;
import java.util.Scanner;

public class Feedback {

    public static void FeedbackandRating() {
        while (true) {
            Scanner sc = new Scanner(System.in);
```

import java.util.Scanner; This import Scanner class from the java.util package. This class is imported to take in user input

public class Feedback A class name Feedback is declared. It is public class meaning it can be accessed from other class

public static void FeedbackandRating() A method name FeedbackandRating is declared within the Feedback class. This method is public and has return type void which does not return any value

while (true) An infinite loop for the carbon method that will always be true unless stop by external intervention

Scanner sc = new Scanner(System.in) A new Scanner object is created inside the FeedbackandRating method. This read input from user input by keyboard, represented by System.in


```
System.out.println(x: "\nIn order for us to enhance your experience on using our application, we would like you\n\nSystem.out.println(x: "Enter your feedback here:");\nSystem.out.println(x: "Enter 'BACK' to go back to Main Page");\n\nString feedback = sc.nextLine();\n\nif ("BACK".equalsIgnoreCase(anotherString: feedback)) {\n    break;\n}
```

- System prints out the sentence introducing what this page is about
- System prints out both the sentence showing user to enter their feedbacks and enter 'BACK' to go back to main page if user don't want to continue
- **String feedback = sc.nextLine();** Reads the user's input assigned for the variable declared as feedback as String datatype
- **if ("BACK".equalsIgnoreCase(userInput))** If user's input is back, ignoring the case, the while loop stopped and bring user back to the main page


```

System.out.println(x: "\nHow much would you like to rate our application\nNote:Please rate between 1 to 5");
int ratings = sc.nextInt();

if (ratings == 1) {
    System.out.println(x: "Very poor");
} else if (ratings == 2) {
    System.out.println(x: "poor");
} else if (ratings == 3) {
    System.out.println(x: "neutral");
} else if (ratings == 4) {
    System.out.println(x: "good");
} else if (ratings == 5) {
    System.out.println(x: "Very good");
} else
    System.out.println(x: "Invalid");

System.out.println(x: "\nThank you for your feedback.");

```

System prints out sentence introducing the feedback feature

String feedback = sc.nextLine(); Reads the user's input assigned for the variable declared as feedback as String datatype

if (ratings == 1) First check if the user enter 1, if input equals to 1, system prints out Very poor

else if (ratings == 2) Then check if the user enter 2, if input equals to 2, system prints out poor and it continue with number 3, 4, 5 that prints out neutral, good, very good accordingly.

else If user enter numbers out of the number range 1-5, system prints out Invalid

At the end, system prints out that sentence after user select the rating

Conclusion



Green Echoes is the best gateway to understanding, mitigating, and adapting to climate change. From an Information Hub offering in-depth insights to real-time data and interactive tools, our application is a dynamic resource empowering users to make informed decisions.



Also, Green Echoes is not merely an application; it is a catalyst for positive change. By imparting knowledge, encouraging sustainable practices, and fostering a sense of global community, we empower individuals to make meaningful contributions to the fight against climate change.



Thank you!

